e is done
- [-] forecast: Iteration 1 of the Kubernetes dashboard pages are enriched with detailed resource spec information

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Goals

- [ ] Iteration 1 is enabled by default
- [ ] Iteration 2 is 50% done and delivered as progresive enhancement over version 1

Milestone 16.10 (February 12, 2024 - March 8, 2024)

Goals

-

Milestone 16.11 (March 11, 2024 - April 12, 2024)

Goals

-

Milestone 17.0 (April 15, 2024 - May 10, 2024)

Goals

-

SECTION: engineering/devops/ops/project-plans/secrets-manager.md

Secrets Manager - Weekly Project Plan

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Create the index page for the secrets manager (Frontend)
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Create form for adding secrets (Frontend)
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin work on backend POC for client authentication

Week of January 29, 2024 (Milestone 16.9)

Team Capacity

- 2 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Finish the MR for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Finish the MR for the add/edit secrets form

Week of February 5, 2024 (Milestone 16.9)

Team Capacity

- 2 FE
- 1 BE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Merge the MR for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Merge the MR for the add/edit secrets form
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin work on backend POC for client authentication

Milestone 16.10 (February 12, 2024 - March 8, 2024)

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/434233 Check that secret exists before viewing details
- https://gitlab.com/gitlab-org/gitlab/-/issues/424460 Create secret details page
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Present backend demo POC for client authentication
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin work on backend POC for GCP idententies

Week of February 12, 2024 (Milestone 16.9)

Team Capacity

- 2 FE
- 1 BE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/434233 Begin work on checking that secret exists before viewing details
- https://gitlab.com/gitlab-org/gitlab/-/issues/424460 Begin work on create secret details page
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Present backend demo POC for client

authentication

Week of February 19, 2024 (Milestone 16.9)

Team Capacity

- 2 FE
- 1 BE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/434233 Complete MR for checking that secret exists before viewing details
- https://gitlab.com/gitlab-org/gitlab/-/issues/424460 Complete MR for create secret details page
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin work on backend POC for GCP idententies

Milestone 16.11 (March 11, 2024 - April 12, 2024)

Goals

-

Milestone 17.0 (April 15, 2024 - May 10, 2024)

Goals

-

Archive

<details><summary>Click to view past plans</summary>

Week of August 14, 2023 (Milestone 16.3 ending)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/421626+ Determine how we want to handle encryption.
- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Create an initial outline for architecture design for backend.
- https://gitlab.com/gitlab-org/gitlab/-/issues/415936+ Create first iteration issue for frontend MVC work.

Week of August 21, 2023 (Milestone 16.4 begins)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/groups/gitlab-org/-/epics/10691 Write up a technical proposal for the POC and create iterative issues.
- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Fill in details on the foundational elements of the architecture design for backend.
- https://gitlab.com/groups/gitlab-org/-/epics/10723 Begin work for first iteration of frontend MVC work.

Week of August 28, 2023 (Milestone 16.4)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Meet with security engineers to discuss our backend architecture proposal.
- https://gitlab.com/groups/gitlab-org/-/epics/11373 Create the frontend issues for the MVC work.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Send out invitations for feedback based on initial UX research about secret names.

Week of September 4, 2023 (Milestone 16.4)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Make adjustments to our backend architecture design based on security feedback.
- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Start the threat model process.
- https://gitlab.com/groups/gitlab-org/-/epics/11373 Finalize the frontend issues for the MVC work.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Report on feedback from interviews.

Week of September 11, 2023 (Milestone 16.4)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Finish first draft of backend architecture design.
- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Work with appsec to continue the threat model process.
- https://gitlab.com/groups/gitlab-org/-/epics/11373 Begin working on the first frontend iteration for the MVC.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Iterate on early designs based on feedback.

Milestone 16.5 (September 18, 2023 - October 16, 2023)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Complete the threat model process.
- https://gitlab.com/groups/gitlab-org/-/epics/11373 Create an MR for the first frontend iteration for the MVC.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Present new design changes.

Week of October 2, 2023 (Milestone 16.5)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-com/gl-security/product-security/appsec/threat-models/-/issues/34 Initialize the threat model process.
- https://gitlab.com/gitlab-org/gitlab/-/issues/424452 Merge MR which creates feature flag and base page for the frontend.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Continue receiving assignment 2 feedback.

Week of October 9, 2023 (Milestone 16.5)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-com/gl-security/product-security/appsec/threat-models/-/issues/34 Complete the threat model process.
- https://gitlab.com/gitlab-org/gitlab/-/issues/424453 Create an MR for creating the root Vue component.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Present feedback findings from assignment 2.

Week of October 16, 2023 (Milestone 16.5 and 16.6)

Team Capacity

- 1 BE
- 1 FE
- 1 Designer

Goals

- https://gitlab.com/gitlab-com/gl-security/product-security/appsec/threat-models/-/issues/34 Address feedback from the threat model process.
- https://gitlab.com/gitlab-org/gitlab/-/issues/424453 Merge MR for creating the root Vue component.
- https://gitlab.com/gitlab-org/ux-research/-/issues/2470 Present new design changes.

Week of October 23, 2023 (Milestone 16.6)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/gitlab-com/gl-security/product-security/appsec/threat-models/-/issues/34 Complete the threat model process.
- https://gitlab.com/gitlab-org/gitlab/-/issues/416701 Create an MR with updated architecture design based on feedback from threat model.
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin working on first backend POC for using GCP key management for key storage.
- https://gitlab.com/gitlab-org/gitlab/-/issues/424452 Merge MR which creates feature flag and base page for the frontend

Week of October 30, 2023 (Milestone 16.6)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/groups/gitlab-org/-/epics/11776 Complete first backend POC for using GCP key management for key storage.
- https://gitlab.com/gitlab-org/gitlab/-/issues/424453 Create an MR for creating the root Vue component

Week of November 6, 2023 (Milestone 16.6)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/groups/gitlab-org/-/epics/11776 Present findings from first backend POC for using GCP key management for key storage.
- https://gitlab.com/groups/gitlab-org/-/epics/11776 Begin work on backend POC for client authentication
- https://gitlab.com/gitlab-org/gitlab/-/issues/424453 Merge the MR for creating the root Vue component

Milestone 16.7 (November 13, 2023 - December 15, 2023)

Goals

- Create the secrets manager index page
- Create the add/edit secrets form Milestone 16.7 (November 13, 2023 - December 15, 2023)

Week of November 13, 2023 (Milestone 16.6)

Team Capacity

- 1 BE
- 1 FE

Goals

- https://gitlab.com/groups/gitlab-org/-/epics/11776 Complete work on backend POC for client authentication
- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Begin work on the secrets managemer index page

Week of November 20, 2023 (Milestone 16.7)

Team Capacity

- 2 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Begin development for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Begin the development for the add/edit secrets form

Week of November 27, 2023 (Milestone 16.7)

Team Capacity

- 2 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Create an MR for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Begin the MR for the add/edit secrets form

Week of December 4, 2023 (Milestone 16.7)

Team Capacity

- 2 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Review the MR for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Review the MR for the add/edit secrets form

Week of December 11, 2023 (Milestone 16.7)

Team Capacity

- 2 FE

Goals

- https://gitlab.com/gitlab-org/gitlab/-/issues/424457 Merge the MR for the secrets manager index page
- https://gitlab.com/gitlab-org/gitlab/-/issues/424458 Merge the MR for the add/edit secrets

form

</details>

SECTION: engineering/devops/ops/project-plans/service-desk-ticket-work-item.md

Use the "Service Desk: Ticket work item type" epic
to follow the progress of this project.

SECTION: engineering/devops/ops/project-plans/ci-catalog/index.md

{{% alert title="Note" color="danger" %}}
The following page may contain information related to upcoming products, features and functionality. It is important to note that the information presented is for informational purposes only, so please do not rely on the information for purchasing or planning purposes. Just like with all projects, the items mentioned on the page are subject to change or delay, and the development, release, and timing of any products, features or functionality remain at the sole discretion of GitLab Inc.
{{% /alert %}}

CI Catalog has been released to General Availability in 17.0.
CI Component and Catalog Product Direction

Current Milestone Goals

- Current milestone goals and focus can be found in our board.

Archive

<details markdown="1">

<summary markdown="span">Past Milestones</summary>

September to October (Milestone 17.5)

Milestone 17.5 (September 14, 2024 - October 11, 2024)

Goals

- Visibility into where components are used - Epic
  - Create fields to return project list where components were used in a pipeline - 466575 (in-dev)
- Security & Compliance workflow epic for CI Catalog
  - Spike + POC Security policies - publish vs usage ci components - 474093
- Release/Publish 2.0 enhancements
  - Add new publish API endpoint with input params - 442783 (in-review)

  - release-cli to extract and validate metadata - 442785
  - Add an indicator if the release goes to the catalog - 438958
- Index/Details page enhancements
  - Update filtering options in CI/CD Catalog index page (Backend part) - 437643
  - Better visualization when the project description is long - 448385
- Search/Filter enhancements
  - Add the by publishing date option to the sort dropdown in /explore - 440508
  - Add an illustration in the search result page - 466412

August to September (Milestone 17.4)

Milestone 17.4 (August 9, 2024 - September 13, 2024)

Goals

- Visibility into where components are used - Epic
  - Create fields to return project list where components were used in a pipeline - 466575 (in-dev)
- Release/Publish 2.0 enhancements
  - Add new publish API endpoint with input params - 442783 (in-review)
- Inputs enhancements
  - Allow interpolation to use local context data - 438275 (in-dev)
  - POC to create JSON schema SSOT for spec keyword - 467375 (complete)

July to August (Milestone 17.3)

Milestone 17.3 (July 13, 2024 - August 9, 2024)

Goals

- Allow administrator to restrict users from publishing to CI/CD Catalog
  - Add GraphQL mutations and types to policy 465265 (blocked by outcome of POC)
- Inputs enhancements
  - Allow interpolation to use local context data - 438275 (in-dev)
  - POC to create JSON schema SSOT for spec keyword - 467375 (in-dev)

June to July (Milestone 17.2)

Milestone 17.2 (June 15, 2024 - July 12, 2024)

Goals

- Index/Details page enhancements
  - Add illustration in the search result page - 466412 (Deferred to candidate::17.4)
  - Add type and description to InputType for Components tab - 466669 (Complete)
- Release/Publish 2.0 enhancements - span multiple milestones
  - Add new publish API endpoint with input params - 442783 (Continuing into 17.3)
- Admin capabilities in CI/CD Catalog - span multiple milestones
  - Add cicomponentsourcepolicy JSON schema - 465264 (Complete)
- Inputs enhancements

  - Allow interpolation to use local context data - 438275 (To start in 17.3)
  - POC to create JSON schema SSOT for spec keyword - 467375 (Continuing into 17.3)

May to June (Milestone 17.1)

Milestone 17.1 (May 11, 2024 - June 14, 2024)

Goals

- Create API to support future requests for badge additions (Complete)
- How to use components from different Cells (In Progress)
  - From recent conversation, determining if CI Catalog support can be deferred to Cells 1.5 at the moment.
- Post-GA follow-ups/technical debt
  - Exclude pre-release from catalog, latest tag, and shorthand fetching (Complete)
  - Add sorting option for prerelease for semver concern (On Hold)
  - Remove ignore rule on inputs and path for catalogresourcecomponents (Complete)
  - GA follow-ups from "Create CI component usage record when component is included in pipeline" - 1 (Complete) and 2 (Complete)
- Tableau component usage visualization work - 1 (Complete) and 2 (Complete)

April to May (Milestone 17.0)

All deliverables for CI Catalog GA are now complete.

- Finish remaining Go-To GA efforts
  - Remove beta label for CI/CD Catalog (Complete)
  - Remove beta label for catalog resource toggle (Complete)
  - Release Process Refinements for GA
    - release-cli pre-GA tasks (Complete)
  - Details page for GA
    - Relative URL breaks CI/CD component catalog project reference (Complete)
    - Fix images not rendering on ReadMe tab (Complete)
- Finishing remaining initial loading badges work
  - Set verificationlevel on publish and fix enum mismatch (Complete)
  - Allow service object to create VerifiedNamespace record (Complete)
  - Create API to support future requests for badge additions
  - NOTE: Initial badge load has been initiated via request

March to April (Milestone 16.11)

Goals

- Go-To GA efforts
  - Inputs for GA (Complete)
    - ~~Change catalogresourcecomponents.inputs to spec~~ (Complete)
    - ~~CI Interpolation for arrays~~ (Complete)
  - Instrumentation for GA (Complete)
    - ~~Table creation for component tracking usage~~ (Complete)
  - Release Process Refinements for GA

    - release-cli pre-GA tasks (In-Verification)
    - ~~Support Self Managed components~~ (Complete)
  - Details page for GA
    - Relative URL breaks CI/CD component catalog project reference (FE work In-Review / BE work complete)
    - ~~Construct component path from parts (stop fetching it from the database)~~ (Complete)
    - ~~Remove the experimental label in the component tab~~ (Complete)
- CI Catalog UX improvements
  - ~~Add badges for components~~ (Complete)
    - NOTE: Backend issue remains before badging starts showing up in CI Catalog.
  - Reorganize information in the detail (In-Review)

February to March (Milestone 16.10)

Goals

- Go-To GA efforts
  - Support Self Managed component to distribute components for Self managed customers. (In Verification)
  - Semantic versioning (Complete)
  - Inputs Enhancements
    - Boolean and number support (Complete)
    - Text interpolation with arrays (In Verification)
  - release-cli pre-GA tasks (FE Complete /BE In-Dev)
- Telemetry instrumentation for GA
  - Implement Tracking for release execution time (Complete)
  - Table creation for component tracking usage (In-Dev)
- CI/CD Components to GA work
  - ~latest returns latest semantic version (Complete)
  - Migrate Versions to follow SemVer convention (Complete)

January to February (Milestone 16.9)

Updates for current Go-To-GA list

- Enforce semantic versioning
  - POC currently in-progress and in review - continue to 16.10
- Support CI interpolation with arrays
  - Implement text interpolation - (Complete)
  - CI interpolation with arrays - To be continued in 16.10 after its prioritized blocker
- Spikes
  - Spike issue to distributed components for Self managed customers.
  - Spike issue to calculate number of times a component is used.
- Threat Model diagrams in-progress to be provided to security. - waiting on security feedback.

Other milestone goals

- Improve the UX for the CI/CD catalog
  - Make star rating default sorting - continue to 16.10 due to capacity
  - Fix Markdown not rendering in CI/CD Catalog (Complete)

- Helper efforts for components
  - Provide components as helpers to test other components - Waiting on product confirmation on prioritization for 16.10
  - Components toolkit to test GitLab-maintained components (Complete)
- Badges
  - Create catalogverifiednamespaces table (Complete)

December to January (Milestone 16.8)

Goals

- Complete initial template to component migration list.
  - AutoDevOps Build component and Test component is complete but discussion on whether Test should exist due to future deprecation.
- Improve UI in the Catalog details page [1, 2]
- Implement Your resource tab in the index page
- Add components tab to the catalog details page
  - BE/FE collaboration needed before feature flag can be rolled out.
- Move inputs to GA ready by completing text interpolation for arrays and !reference.

November to December (Milestone 16.7)

CI Catalog - Pages & Navigation

- 100% complete, Complete, Delivered in 16.7
- Status: As of 2023-12-08, last MR is merged to make Catalog available in explore navigation permanently.

CI Catalog - Search & Filter

- 100% complete, Complete, Delivered in 16.7
- Status: All Beta work is in production.
- Risks/Blockers: Beta work complete.

CI Catalog - Release Process refinements

- 100% complete, Complete, Delivered in 16.7
- Status: On 2023-12-01, the toggle back and forth is complete is now in production to complete all Beta work.
- Risks/Blockers: Beta work complete.

October to November (Milestone 16.6)

- CI Catalog - Pages & Navigation
  - x] [Move shared components to Free Tier
  - x] [Add route and nav for Global CI/CD Catalog
  - x] [Coordinate with Foundations on adding Global catalog to the Explore navigation
  - x] [Make the README tab the default view for component
  - x] [Add an indicator to the catalog resource project
  - x] [Prepare Ci::Catalog::Listing for global CI Catalog

  - x] [Move GraphQL Catalog code to FOSS
  - x] [Remove fork count from Catalog details page
  - x] [Add a copy-to-clipboard button in the code snippet in the component tab
  - x] [Empty state when there is no metadata for the components details
  - x] [Add global Catalog arguments to GraphQL
  -] [Add Vue application for Global page ~workflow::in review
  -] [Make ciCatalogResource accept a fullpath argument ~workflow::in dev
  -] [Add components field to ciCatalogResource ~workflow::in dev
  -] [Add the new components tab
  -] [FF rollout cicatalogcomponentstab
  -] [Add namespace to scope for Catalog resources query
  -] [Remove projectPath argument from ciCatalogResources
- CI Catalog - Search & Filter
  - x] [Add GraphQL search filter and sort by createdat to ciCatalogResources
  - x] [Create database indices for CI Catalog
  - x] [Denormalize name and description in Ci::Catalog::Listing
  -] [Add Search bar ~workflow::in review
  -] [Update catalogresource.latestreleasedat when version is created/deleted
- CI Catalog - Release Process refinements
  - x] [Add mutation to mark a catalog resource as draft
  - x] [Add path column where to persist full path to component YAML file
  - x] [Update the docs to reflect the recommended method for the release
  - x] [Fix regexp to scan for component files
  - x] [Scanning components on release and collect metadata
  -] [Update the releases logic in catalog resources to use the Version association ~workflow::in review
  -] [Create a migration to update state and add metadata to existing catalog resources
- Completion of [Inputs Enhancements]
  - x] [Support options: with inputs defining default: value

 Week of October 2, 2023

 Goals

- Frontend CI Catalog Details page work
- Scanning components on release and adding release sort

 Week of September 25, 2023 (Milestone 16.5)

 Team Capacity

- 3 Backend Engineers (Leaminn, Avielle, Laura)
- 1 Frontend Engineer (Frédéric)

 Goals

- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/387632 to add support for variables ininputs: syntax so when expandvars is used, error is raised due to security reasons. ~workflow::in review
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/408382 to add released sort to CI Catalog.

-] [https://gitlab.com/gitlab-org/gitlab/-/issues/411438 to support CI interpolation with arrays.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415413 to scan a catalog resource for components on release creation.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/424962 to put the current right side column in the Catalog header. - ~workflow::in review
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/424966 to add the README tab with the current content.

Week of September 18, 2023 (first week of Milestone 16.5)

Team Capacity

- 4 Backend Engineers (Avielle, Laura, Kasia, Leaminn)
- 1 Frontend Engineer (Fred)

Goals

-] [https://gitlab.com/gitlab-org/gitlab/-/issues/387632 to add support for variables ininputs: syntax so when expandvars is used, error is raised due to security reasons. ~workflow::in review
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415413 to scan a catalog resource for components on release creation.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/408382 to add released sort to CI Catalog.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/424962 to put the current right side column in the Catalog header - ~workflow::in review
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415637 to add an optional description field under input - handled by Community Contributor

Week of September 11, 2023 (last week of Milestone 16.4)

Team Capacity

- 2 Backend Engineers
- %16.4 security priorities are nearly complete so more BE focus is shifting in later %16.4

Goals

-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415413 to scan a catalog resource for components on release creation.
-] Spike follow-up to [https://gitlab.com/gitlab-org/gitlab/-/issues/411438

Week of September 4, 2023 (Milestone 16.4)

Team Capacity

- .5 Backend Engineers
  - Due to %16.4 security priorities, the weekly goals list will be shorter due to our focus there.
  - Working on https://gitlab.com/gitlab-org/gitlab/-/issues/423456 for feature addition for

needs: parallel: matrix enhancements.

Goals

- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/418996 for marking catalog resource as draft, if final version removed.

Week of August 28, 2023 (Milestone 16.4)

Team Capacity

- 1.5 Backend Engineers
  - Due to %16.4 security priorities, the weekly goals list will be shorter due to our focus there.
  - Working on https://gitlab.com/gitlab-org/gitlab/-/issues/423456 for feature addition for needs: parallel: matrix enhancements.

Goals

- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/411394 for adding instrumentation for number of components are used.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/418996 for marking catalog resource as draft, if final version removed.

Week of August 21, 2023 (Milestone 16.4 begins)

Team Capacity

- 3 Backend Engineers

Goals

-] [https://gitlab.com/gitlab-org/gitlab/-/issues/411394 for adding instrumentation for number of components are used.
- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/415853 for updating CI component fetching for updated directory structure - to be merged this week.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/409846 work continues for creating an SSOT for CI config loading
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/411438 CI interpolation with arrays spike work continues.

Week of August 14, 2023 (Milestone 16.3 ends)

Team Capacity

- 3 Backend Engineers
- 2 Frontend Engineers

Goals

- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/409041 for showing pipeline status for latest version of catalog resource.
- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/415287 for creating catalogresourcecomponents table to unblock other issues.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/412948 for updating permissions for namespace catalog & update resolver
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/409846 for complete last MR for CI config loading

Week of August 7, 2023 (Milestone 16.3)

Team Capacity

- 1.5 Backend Engineers
- 2 Frontend Engineers

Goals

- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/418785 for moving CI Catalog to be a premium feature.
- x] [https://gitlab.com/gitlab-org/gitlab/-/issues/390458 for input type validation.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/409041 related to showing pipeline status for latest version of catalog resource.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415287 for creating catalogresourcecomponents table to unblock other issues.
-] [https://gitlab.com/gitlab-org/gitlab/-/issues/415853 for updating CI component fetching for updated directory structure.

</details>

SECTION: engineering/devops/ops/verify/_index.md

Vision

We enable global software organizations and teams to make great decisions with smart feedback loops by delivering speedy, reliable pipelines in a comprehensive CI platform that embodies Operations for All.

Technical Roadmap

FY25 to FY26

There are 3 Core Themes that continue to be a focus for GitLab CI:

1. Scalability
1. Availability
1. Sustainability & Maintainability

We aim to drive enhancements that benefit GitLab.com, Self-Managed and Dedicated customers.

1. With accelerated efforts, we aim to complete CI Data Partitioning for our top 6 tables in the CI Database in FY25. Not only does this impact CI, this addresses a critical availability need impacting all of gitlab.com. This is a cross-stage effort with backend engineers across Verify in order to parallelize the ongoing effort. (ETA: Q3)
1. CI Data Retention: with the continued growth in our CI database tables, Verify and Infrastructure teams will work on removing data upon analysis of disk usage. This includes removal of both table records and indexes. Engineering will also collaborate with Product to implement features that allow Self-Managed and Dedicated customers to configure their own CI data retention policies. (ETA: Analysis in Q3, Implementation in Q4 until FY26-Q1/Q2)
1. CI Data Management (TBD): Determine the data management tools that provide the greatest flexibility to our customers to manage their own CI data. (For example, removal of old CI builds or artifacts that consume their disk usage). We will consult with Product and UX to understand the tools/feature set that are most requested. (ETA: FY26-Q3)
1. CI Minutes / Compute Units support: Build better tooling for our Support and SRE teams when responding to incidents that require us to restore CI Minutes for customer namespaces on GitLab.com, and ensure domain knowledge is shared across the Verify and Fulfillment teams. (ETA: Q4)
1. Pipeline creation speed improvements: benchmarking and instrumentation will be our focus in order to identify the bottlenecks and improve pipeline creation speeds. This will be critical for driving Error Budget improvements. (ETA: Ongoing efforts to FY26)
1. Cells 1.0 Database Support: the goal is to complete the Verify dependencies by Q4.

The product roadmap outlines the expected deliverables for FY25.

3 Year Vision

1. Continue to iterate on the Verify stage technical debt roadmap. How do we represent the highest priorities in a SSOT like an issue board?
1. Tracking Unplanned Work and making that more visible, to account for this during Engineering capacity or headcount planning. (For example, incident response, requests for help issues, triaging questions on Slack)
1. Pipeline speed improvements - while benchmarking and instrumentation has been a focus, we have not:
    1. Implemented distributed tracing on our CI workers
    1. Prioritized further work in CI/CD Build Speed working group
    1. Built more observability related to Pipeline speed improvements.
1. CI Events - this is deferred from FY25 due to capacity constraints in Verify.
1. Better onboarding for contributors who are not CI subject domain experts - this includes improvements to code readability and accessibility, or better documentation and onboarding material.

FY24

The Verify Pipeline teams focused on the following Engineering-led initiatives, in addition to our deliverables for the FY24 Yearlies:

1. CI Data Partitioning
1. Pipeline speed improvements - including analysis of pipeline performance

1. Review of the data retention strategy of CI data on gitlab.com
1. Security vulnerabilities and infradev issues related to SaaS availability
1. S1/S2 bug burndown of Categories that do not have planned feature development for FY24.
    1. Note that this also includes the Continuous Integration category, which has the biggest backlog of bugs in Verify. While it may be considered to be "Maintenance" (no new feature development planned), this work remains critical in ensuring we keep GitLab CI performant and reliable.
    1. Pipeline Execution owns the Continuous Integration category. The team is also the DRI for CI Data Partitioning and Pipeline speed improvement efforts.

FY23

The Verify stage focused on reliability and scalability of GitLab CI, which was critical for the availability of gitlab.com. This included addressing database performance, security vulnerabilities, performance improvements and relevant technical debt.
This ensured GitLab remained secure, compliant and performant, with our SaaS offering that was able to maintain SLAs of gitlab.com.

Mission

As engineers in Verify we know our customers because we are our customers, and we are constantly striving to make our platform better for everyone. We do this through iteration, dogfooding, and being involved in our open source community. We innovate, we collaborate, and we challenge assumptions to arrive at great results.

We take ownership of the things we build, with a focus on stability and availability. We do this by having a deep technical understanding of the operation and performance characteristics of our platform, and a proactive perspective to future growth.

Who we are

The Verify stage is made up of 5 groups:

1. Verify:Pipeline Authoring

1. Verify:Pipeline Execution

1. Verify:Runner

1. Verify:CI Platform

Verify:Pipeline Authoring

{{< team-by-manager-role "Engineering Manager(.+)Pipeline Authoring" >}}

Verify:Pipeline Execution

{{< team-by-manager-role "Engineering Manager(.+)Pipeline Execution" >}}

Verify:Runner

{{< team-by-manager-role "Engineering Manager(.+)Runner" >}}

Verify:CI Platform

{{< team-by-manager-role role="Senior Manager(.+)Verify" team="CI Platform" >}}

Verify Engineering Leaders

{{< team-by-manager-role role="Senior(.+)Manager(.+)(Verify)" team="(Principal|Pipeline)">}}

Stable Counterparts

{{< engineering/stable-counterparts role="Verify" other-manager-roles="Engineering Manager(.+)(Pipeline Authoring|Pipeline Execution|Runners)|Senior Manager(.+)Verify" >}}

How we work

Jobs to be done (JTBD)

A Job to be Done (JTBD) is a framework, or lens, for viewing products and solutions in terms of the jobs customers are trying to achieve.

Verify:Pipeline Execution JTBD
Verify:Pipeline Authoring JTBD
Verify:Runner JTBD

Developer Onboarding in Verify

Welcome to the team! Whether you're joining GitLab as a new hire, transferring internally, or ramping up on the CI domain knowledge to tackle issues in our area, you'll be assigned an onboarding/shadowing buddy so you can have someone to work with as you're getting familiarized with our codebase, our tech stack and general development processes on your Verify team.

Read over this page as a starting point and feel free to set up regular sync or async conversations with your buddy. We recommend setting up weekly touch points, at a minimum, and joining our regular team syncs to learn more about how we work. (Reach out to our Engineering Managers for an invite to those recurring meetings). Please also schedule a few coffee chats to meet some members of our team. You will be assigned a team specific developer onboarding issue (For example, Pipeline Execution Developer onboarding checklist) for you to go through. It contains admin tasks to complete (as a new team member, if relevant), and also links to technical documentation, meeting agendas, and recordings.

Issues labeled with ~onboarding are smaller issues to help you get onboarded into the CI feature area. We typically work Kanban, so if there aren't any ~onboarding issues in the current milestone, reach out to the Product Manager and/or Engineering Managers to see which issues you can start on as part of your onboarding period.

In May 2021, we introduced the CI Shadow Program, which we are trialing as a way to onboard existing GitLab team members from other Engineering teams to the CI domain and contribute to CI features.

Onboarding Buddies

Onboarding buddies are assigned to new hires to ensure their first few months of onboarding go smoothly. It's recommended that onboarding buddies set up weekly check-ins, whether that's async (such as a Slack DM) or sync (such as recurring coffee chats).

Reviewing Merge Requests

In addition to helping those new hire/transfer through any issues with their set up or assigned tasks, it's recommended that their onboarding buddies add the new hire/transfer as an additional reviewer on any MRs the onboarding buddy has been requested to review. Ideally this takes place after they've been working in Verify for at least 3 months, and as mutually agreed upon between both parties. This step further builds the new hire/transfer's CI knowledge and allows for CI domain expertise to be shared amongst all engineers in Verify.

Similar to our reviewer mentorship programs, the new hire/transfer will review the merge request as if they're being asked to perform the code review. Once complete, they'll assign the MR back to their onboarding buddy. It is expected that the onboarding buddy will also complete the code review, then provide the new hire/transfer feedback about their code review. Ideally this takes place at their next check-in, where notes are captured in a shared Google Doc or a GitLab issue for ease-of-collaboration.

API development

Our API exists in two formats (REST and GraphQL) which should allow the same degree of querying.
In Verify, we are GraphQL first which means that we will develop new user facing features using GraphQL by default.
We will refactor older REST user facing features to support GraphQL wherever possible.
In some instances, it might make more sense to keep or even develop a new feature using REST. For example, REST is better at handling files than GraphQL so it might be better to preserve this functionality in REST.
We allow each team to decide when they think they should go with REST, but eventually the goal is to have everything in GraphQL.

Shared issues

In the Verify team we lean in to the GitLab mission, "everyone can contribute"!
To help balance this workload out across the groups, we use the Verify candidate label.
Every issue with this label is a good candidate to be worked on by any group in the Verify stage.
This applies to both frontend and backend issues.
Prioritization is still determined by the Product Manager, such as ensuring any deliverables in the engineer's own group still take priority, but engineers are encouraged to pick work up from this board.
This helps us break down silos, balance the workload, and prevent disruptive re-allocations.

To help with prioritizing within the list of available Verify candidate issues, it's recommended to reference the issue types in the Product Priorities list, noting any severities applied on the issues as well.

Shared technical debt

In the Verify stage, prioritizing our technical debt so that we can move faster is a top priority. Starting in August 2024, the Pipeline teams in the Verify stage have created a board for Pipeline teams in the Verify stage that is intended to help us prioritize technical debt and bring alignment across team members of what is the most critical technical debt work to be focusing on. An engineer DRI from each team will work closely with their EM to align on which issues should be advocated for the most at a given time.

Issue Health Status Definitions & Async Issue Updates

Across Verify we value Transparency, we live our values of Inclusion, and we expect Efficiency by dogfooding using Issue Health Statuses and providing regular updates on active issues. Each team in the Verify Stage will define the cadence of updates and specific definition of the statuses, but generally the expectation is a weekly update on in progress issues with the following Health Statuses:

On Track
Needs Attention
At Risk

These updates are an opportunity for the engineer to add detail to the status and are not expected to provide a justification for why something is behind or will miss a milestone. We encourage blameless problem solving and kindness at all times.

Merge Requests in Verify

In the Verify stage, we value MR Rate as a shared performance indicator for team collaboration, iteration, customer results. The entire team is responsible for iterative scope in issues. This starts with product management creating a clear problem statement connected to user insights. UX then adds interaction specifications and acceptance criteria to then be considered and weighed by the engineering team. Teams are encouraged to iterate on scope so as to delivery the smallest thing possible.

By considering MR Rate as a measure of throughput, product management is focused on creating decomposed pieces of scope to improve the user experience. This encourages the UX and engineering teammates to provide simpler ways to solve the same problem, ultimately improving the throughput of the entire team.

Since April 2023, code changes to Verify code require approval from a Verify maintainer since Continuous Integration platform overall is a critical GitLab feature.
In order to track quality of the approval process
we ask Verify maintainers to apply one of the following labels to a merge request changing Verify code:

~"verify-review::impacted" for merge requests where the maintainer was able to identify near miss bugs, inefficiencies and tech debt.
~"verify-review::not impacted" for merge requests where the change was trivial
or no issues were found by the Verify maintainer.

Pipeline Authoring and Pipeline Execution Collaboration

Pipeline Authoring and Pipeline Execution are closely related but they also represent different stages in the cycle of a user's interaction with a pipeline. At a very high-level, this image illustrates the main focus of each group and how they can both support a better pipeline experience.

Async Work Week

We have quarterly async work weeks in Verify that start on the first Monday of the quarter.

Some of the noted benefits include reduced time spent in sync meetings, allowing for more focus that aligns with our async-first communication and our Diversity and Inclusion value to bias towards more async communication. However, this doesn't preclude us from having any meetings; it's up to the respective meeting attendees to decide accordingly. Exceptions might include: high priority issues and initiatives, social calls, coffee catch-ups. This also does not mean that you should not default to async-first at other times. Having regularly scheduled async weeks ensures that our processes do not become dependent on synchronous meetings.

Verify Engineering - Async Updates

Current (2022 onward)

As of June 2022, async issue updates are created weekly at the stage level and for each of the groups within the stage, following the Ops section process of async updates. Contributions will be added by Principal+ Engineers, Engineering Managers, and the Senior Engineering Manager of the Verify stage.

2020-2021

Every two weeks the Verify Engineering Update Newsletter is set out to an opt-in subscriber list. The purpose of the email is to share recent highlights from the Verify stage so folks will have a better idea of what is happening on other teams, and provide new opportunities for learning and collaboration.

Everyone is welcome to sign up or view previous issue on the newsletter page (link no longer available).

Each issue of the newsletter is planned using individual issues linked in the newsletter epic. Content is generally contributed by managers, but everyone is encouraged to contribute topics for the newsletter.

Verify Technical Discussions

Verify Technical Discussions is a Zoom meeting hosted monthly by the team members in the Verify stage. Everyone is invited, however participation from the Verify stage members is especially encouraged.

During the meeting we discuss a variety of technical aspects related to the Verify stage roadmap. Folks are also encouraged to challenges they're facing working on problems in the CI domain.

Everyone can add their points to the agenda document.

Below you can find a table with links to recordings of Verify Technical Discussions and Technical Deep Dives.

Current Inventory of Recordings

The Verify Technical Discussions are automatically recorded and added to Google Drive (internal).

Uploaded Recordings to YouTube

Date	Title	Recording
2021-01-21	Technical Discussions - Pipeline Editor and database storage	Recording
2021-01-07	Technical Discussions - Next iteration of CI/CD Pipeline DAG	Recording
2020-12-10	Technical Deep Dive - Observability at GitLab with demos	Recording
2020-11-19	Technical Deep Dive - Cloud Native Build Logs feature overview	Recording
2020-05-08	Technical Deep Dive - Using Prometheus with GitLab Compose Kit	Recording

Weekly Triage Reports

The Product Manager, Engineering Manager(s), and Designer for each group are responsible for triaging and scheduling feature proposals and bugs as part of the Weekly Triage Report. Product Managers schedule issues by assigning them to a Milestone or the Backlog. For bug triage, Engineering Managers and Product Managers work together to ensure that the correct severity and priority labels have been applied. Since Product Managers are the DRIs for prioritization, they will validate and/or apply the initial priority label to bugs. Engineering Managers are responsible for adding or updating the severity labels for bugs listed in the triage report, and to help Product Management with understanding the criticality and technical feasibility of the bug, as needed.

While SLOs for resolving bugs are tied to severities, it is up to Product Management to set priorities for the group with an appropriate target resolution time. For example, criteria such as the volume of severity::2 level bugs may make it appropriate for the Product Manager to adjust the priority accordingly to reflect the expected time to resolution.

Availability, security, performance, and bug triage process

In the Verify Stage, we aim to solve new availability, security, performance issues within the SLO of the assigned severity. These types of issues are the top priority followed by bugs and technical debt according to our severity SLO chart.

Availability and performance issues (commonly referred to as infradev) are also triaged in the

Infra/Dev Triage Board.

Supporting Community Contributions

We believe in supporting our open source community. We aim to support two main measure of success:

1. Merged MRs from community contributions
1. MRARR

Each team in the Verify Stage follows roughly the same process to ensure the community is effectively supported and free to add features or fixes to the product. How we manage the Community Contribution MRs is spread across three main areas: processing the contributions, reviewing the contributions, and merging the contributions.

Process

Code contributions to Verify typically occur in three flavors:

1. Free users, open source contributions from already scoped issues
1. Paid users, open source contributions not from scoped issues
1. Paid users, proprietary contributions not from scoped issues

Contributions from both free and paid users are equally important and will follow our GitLab Contribution Guidelines. We strive to make this process as frictionless as possible between our users and the Engineering teams in Verify, especially during the reviewing of contributions.

Reviewing contributions

Once a contribution has been created, the Engineering Manager assigns an engineer to manage and review with the Community Contributor. Reviewing contributions will follow the definition of done, style guidelines, and other practices of development. As the DRI of the review, the assigned Verify engineer will work with Community Contributor to resolve any outstanding items. The MR is then passed to a Maintainer with relevant domain expertise for final review prior to being merged.

Contributions from Partners

Our partners are an important part of our ecosystem at GitLab. These contributions should be reviewed with the same GitLab Contribution Guidelines as community MR contributions, and aligns with the Verify contribution guidelines for working in the Verify areas of the codebase.

Merging the Contribution

The Maintainer of the codebase will be the DRI of merging the contribution into the Verify product.

SLOs for Community Contributions

For issues that are refined, they are considered priority for review and merging. Refined

issues are defined as issues in workflow::ready for development, with direction labels, and either have technical proposals or are weighted. Issues that are not labeled with workflow::ready for development and direction labels are considered non-refined and are lower priority for review, and therefore have longer merge SLOs. SLOs for these two types of issues are defined in the table below:

Types of issues & users	Time Frame for Review SLO	Time frame for Merge SLO
All users contributing to refined issues or bugs of severity S2 or S1	30 days	The next release (60 days)
Paying users from non-refined issues or bugs of severity S3	60 days	Within the next 3 releases (approx one quarter or 90 days)
Non-paying users from non-refined issues or bugs of severity S4	90 days	Anything outside the next 3 releases (more than one quarter or 120 days).

In order to prevent the inflow of Community Contributions overwhelming engineers on the team and impacting their ability to work on planned issues, there is a WIP limit of 5 assigned Community Contribution MRs per reviewing engineer. This helps limit the amount of context switching a single engineer is doing and prevents them from being overwhelmed with reviews.

Managing urgent priorities

In the event of escalations or high priority tasks that come up, we will adopt a follow-the-sun rotation to ensure that the task is completed as quickly as possible. Examples of these urgent tasks include (but are not limited to): critical customer fixes, high severity security issues or corrective actions related to a high severity incident, with this level of urgency to be confirmed by the team's leadership (e.g. Engineering Manager and Product Manager). The follow-the-sun rotation requires an engineer to focus on that task within their working hours, then transitions to the next engineer once they come online, and engineers will arrange handoff of work to one another as needed (for example, when collaborating on an MR together as the contributor and reviewer, respectively). This effort can also be a cross-stage effort, which involves collaboration amongst engineers across Verify stage groups.

As part of our GitLab values, we strive to be inclusive to those in regions with fewer employees. As a result, there is no expectation that people are expected to continuously work outside of their regular business hours. In the event we have limited people available in certain regions (such as APAC), we should look to escalate outside of the Verify stage to maintain focus on the escalated effort, by reaching out to engineers or other SMEs, and have managers help with this escalation path as needed.

Common Links

Slack Channels

Verify sverify
  Verify:Pipeline Execution gpipeline-execution
  Verify:Pipeline Authoring gpipeline-authoring
  Verify:Runner grunner
  Verify:Pipeline Security gpipeline-security
Verify Engineering Management sverify-em

Verify Frontend Engineering sverifyfe
CI/CD UX uxci-cd

SECTION: engineering/devops/ops/verify/ci-platform/_index.md

Vision

Our team is responsible for the following categories:

- CI Scaling

Mission

- Ensure that GitLab Continuous Integration is scalable, performant and reliable, by improving the resiliency of our product, as well as future proofing the CI platform as we grow.
- Support database and infrastructure initiatives that our CI Platform requires to remain operational.

Team Members

The following people are permanent members of the Verify:CI Platform group:

{{< team-by-manager-role role="Senior Manager(.+)Verify" team="CI Platform" >}}

Useful Links

- Issue tracker: ~group::ci platform
- Slack channel: gci-platform
- Issue board: CI Platform Workflow

Dashboards

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="ci platform" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="ci platform" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="ci platform" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="ci platform" >}}
{{< /tableau >}}

How we work

Workflow

TBD

Planning

TBD

Async Collaboration

Every other day

We use geekbot integrated with Slack for our every-other-day async standup to make our current work and upcoming work more visible.

Weekly

Given the distributed nature of our team members, our "team meetings" consist of collaborating async through a Google Doc Agenda (internal). Slackbot will remind the team weekly to review the agenda and encourage us to collaborate through the agenda.

Monthly

We use a GitLab issue in the (internal) async retrospectives project for our monthly retrospective. The purpose of the monthly retrospective issue is to collaborate async and reflect on how the milestone went, in terms of:

- what went well
- what didn't go so well
- what we can do improve upon
- what praise we have for one another.

Instead of waiting until the end of the milestone to add items to the retrospective issue, we encourage team members to add comments throughout the month. The issue's due dates and Slackbot reminders serve as reminders to contribute feedback to our issue prior to closing.

Labels

Category Labels

The CI Scaling group supports the feature categories described below:

Label				
Category:Continuous Integration (CI) Scaling	Issues	MRs	Direction	Documentation - TBD

Developer Onboarding

Refer to the Developer Onboarding in Verify section.

SECTION: engineering/devops/ops/verify/pipeline-authoring/_index.md

Useful Links

Product

- Product Vision
- Pipeline Authoring Category direction
- number of unique users who trigger cipipelines (Performance indicator) - Internal link
- CI/CD Development Documentation
- CI/CD Components Documentation
- CI/CD Catalog Documentation

Team Handles

Category	Handle
GitLab Team Handle	@verify-pa-team
Slack Channel	gpipeline-authoring
Slack Handle (Engineers)	@verify-pa-engineering

Team Resources

- Team Resources
- Workflow board: ~group::pipeline authoring

Videos

- GitLab Unfiltered: Pipeline Execution group (CI Related)
- CI Backend Architectural Walkthrough - May 2020
- Frontend CI product / codebase overview - June 2020
- CI/CD Catalog Demo

Core domain

Products

Product	Navigation	Documentation

CI/CD Pipelines	Build >> Pipelines	GitLab docs
Pipeline editor	Build >> Pipeline editor	GitLab docs
CI/CD Catalog	Explore >> CI/CD Catalog	GitLab docs

Features

- Pipeline creation
- YAML syntax
- CI/CD configuration lint tool
- CI/CD Variables
- Additional features can be found here

Technical Roadmap

Remainder of FY25

These areas are our high-level engineering driven goals for the remainder of FY25. Though they are ambitious and subject to change, it gives insight into where our focus will be in these areas.

Performance & Cost Reduction

Pipeline Creation Speed

NOTE: Pipeline creation feedback issue. We welcome your thoughts on your experience with the pipeline creation process.

Goals:

- Understand the components of the pipeline creation service to improve speed.
  - This graph captures data around the pipeline creation performance.
- Top 3 identified issues to optimize pipeline creation performance.
  - Bulk insert Sidekiq jobs when creating a pipeline
  - Expand CI variables lazily and selectively
  - Evaluate rules once for parallel jobs

Efficiency

Goals:

- Transition from entry classes to JSON schema for data structure validation. - Issue

Visibility

CI Catalog instrumentation

Goals:

- Top 3 identified issues to add more visibility for usage in CI/CD Catalog with adding instrumentation.

- Capture GraphQL call times
- Implement instrumentation to track performance of Ci::Catalog::Listing queries
- Capture page load times

FY26 top of mind

- Support for GraphQL subscriptions - epic
- Pipeline performance improvements - epic
- CI Variables improvements - epic
- MR Pipeline Tab migration to GraphQL - epic

Exciting things and accomplishments

This section will list the top three most recent, exciting accomplishments from the team.

- Recently, we released the CI/CD Catalog to GA.
- We added usage statistics and a sort option for popularity in the CI/CD Catalog index page

Team Members

{{< team-by-manager-role "Engineering Manager(.)Verify:Pipeline Authoring" >}}

Stable Counterparts

To find our stable counterparts, look at the Pipeline Authoring product category listing.

Capturing User Feedback

We highly value user feedback! Please use the issue below to capture feedback and insights for our newest feature, CI/CD Catalog:

- CI/CD Catalog Feedback

Community Contributions

Pipeline authoring at GitLab involves a deep understanding of our CI/CD system and its intricate interactions with various components. This complexity can present challenges for those unfamiliar with the codebase and underlying architecture.

We encourage community contributors to work on issues with the ~"seeking community contribution" label. These issues are specifically selected to provide a clear and manageable path for external contributions.

Due to the inherent complexity of pipeline authoring, contributions to issues not marked for community contribution may require significant domain knowledge and familiarity with GitLab's internals. While we appreciate all contributions, Merge Requests for these issues may require more extensive review and may not be merged if they don't align with our architectural vision or best practices.

Guidance for Reviewers

When reviewing Merge Requests from community contributors, please consider the following:

- Issue Selection: Verify if the contribution addresses an issue labeled ~"seeking community contribution". If not, assess the complexity and required domain knowledge.
- Mentorship: If the contribution shows promise but needs guidance, provide constructive feedback and consider offering mentorship to help the contributor succeed.
- Clear Communication: Explain the rationale behind any required changes or decisions clearly and respectfully, ensuring the contributor understands the context and reasoning.

If the issue picked up by the community does not have the ~'seeking community contribution' label and/or is too complex please consider using the following template:

markdown
"Thank you for your interest in contributing to GitLab! We appreciate you taking the initiative to work on this. This particular area involves complexities that may require deeper domain knowledge of GitLab's CI/CD system. To ensure a smoother review process and increase the likelihood of your contribution being merged, we recommend focusing on issues with the ~'seeking community contribution' label. These are specifically curated for external contributors. A curated list of issues is available here: https://gitlab.com/gitlab-org/gitlab/-/issues/?sort=popularity&state=opened&labelname%5B%5D=group%3A%3Apipeline%20authoring&labelname%5B%5D=Seeking%20community%20contributions&firstpagesize=100"

Group Meetings

For every meeting, it is expected that it contains a list of agenda items to discuss along with the meeting notes when the meeting takes place. This will make it easier for people to catch up if they are unable to attend.

Please note that sync meeting schedules are flexible and can be moved to accomodate required participants. For an up-to-date schedule of all team meetings, please consult the Group's Calendar.

The table below briefly outlines the objectives and key details of regular team meetings:

Meeting Title	Cadence	DRI	What
Team weekly sync	Weekly	Everyone	Discuss everything team related, meeting rotates across timezones each week.
Design discussions	Bi-Weekly	UX	Review current design work that needs collaboration or feedback from Engineering.
Technical discussions	Bi-Weekly	Engineers	Discuss current work and open questions that team members have for each other

Dashboards

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
{{< tableau/filters "GROUPLABEL"="pipeline authoring" >}}

{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="pipeline authoring" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="pipeline authoring" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="pipeline authoring" >}}
{{< /tableau >}}

Cross-functional prioritisation

UX, Product Manager and Engineering Manager meet weekly to discuss cross-functional prioritisation in addition to any other topics that require the quad to collaborate on.

Design Collaboration

We hold a bi-weekly design sync meeting open to all team members where we discuss any design-related topic.

How We Work

Planning

Issues are refined and weighted prior to assigning them to a milestone. We use candidate:: scoped labels to help with planning work in future milestones. This label allows us to filter on the issues we are planning, allowing Product, Engineering, and UX to refine issues async that have workflow::design and workflow::ready for development labels applied. Weighting also helps with capacity planning with respect to how issues are scheduled in future milestones.

We create a planning issue as part of our milestone planning process and the workflow board is the single source of truth (SSOT) for current and upcoming work. Product is the DRI in prioritizing work, with input from Engineering, UX, and Technical Writers. The planning issue is used to discuss questions and team capacity. Prior to the beginning of each milestone, issues identified in the planning issue will be assigned to that milestone and engineers can assign prioritized issues to themselves from the top of the workflow::ready for development column in the workflow board.

Finding issues that need refinement

Issues that needs to be refined are listed in the planning issue associated with the upcoming milestone(s)

How Engineering Refines Issues

side note: we prefer using Refining over Grooming

The purpose of refining an issue is to ensure the problem statement is clear enough to provide a rough effort sizing estimate; the intention is not to provide solution validation during refinement.

Checklist for Refining Issues

1. Does the issue have a problem statement in the description?
1. Does the issue have the expected behaviour described well enough for anyone to understand?
1. Does the issue explicitly define who the stakeholders are (e.g. BE, FE, PM, UX and/or Tech Writer)?
1. Does the issue have a proposal in the description? If so:
    1. Does the proposal address the problem statement?
    1. Are there any unintended side effects of the implementation?
1. Does the issue have proper labeling matching the job to be done? (e.g. bug, feature, performance)
1. If the issue is a type::bug, is it clear how to reproduce the behavior and can a sample CI configuration file be provided?

Any one on the team can contribute to answering the questions in this checklist, but the final decisions are up to the PM and EMs.

Use of Sub-tasks

We use sub-tasks in issues to help with refinement. The goal is that we have one SSOT issue that contains threaded discussions around all aspects of the feature which include backend, frontend, UX, Product, and documentation.

Steps for Refining and Weighting Issues

Engineers will:

1. Go through the checklist above for refining issues they assign to themselves prior to each milestone.
1. Add a weight based on the definitions.
1. Update the ~workflow:: label to the appropriate status, for example:
    - ~"workflow::design" if further design refinement is needed, and let the designer know.
    - ~"workflow::ready for development" when refinement is complete and a weight has been applied, signaling that it's ready for implementation and the issue can be scheduled accordingly.
    - ~"workflow::planning breakdown" if more investigation or research is needed, the status does not move, and the PM and EMs should be pinged.
1. Unassign themselves from the issue when they are done refining and weighting the issue.

Weighting Issues

The weights we use are:

Weight	Extra investigation	Surprises	Collaboration
1: Trivial	not expected	not expected	not required
2: Small	possible	possible	possible
3: Medium	likely	likely	likely
5: Large	guaranteed	guaranteed	guaranteed

The above table is contextual. Domain knowledge, experience level, and time at GitLab can impact an engineer's perspective on whether an issue requires extra investigation or surprises.

Weights are not set in stone. We do our best to get it righ